

5주 차 보고서

(2026년 1 월 25일 ~ 2026년 1 월 31일)

작성자

박신우

이번 주 회의 내용

- 차량 기관총,바주카포,총알,차량가시,차량 강화 부품. 어떤 에셋을 사용할지
- 윤진이 맵에 대한 의견 공유.
- 인벤토리 UI
- 맵 만들어 main에 merge 진행일정 조율
- 각자 진행상황 공유

이번 주 한 일

01.26 트럭 사운드

저번에 트럭이 운전할 때 소리가 변하지 않는 문제가 있었는데

RPM 값이 계속 1200 인근에서 맴도는 문제가 있었다.

트럭의 무게 값을 너무 높게 설정해서 RPM값이 잘 오르지 않았던 것 같다.

무게값을 조정해 주니 RPM이 잘 변하였고 속도에 따른 속도 변화도 이루어졌다.

1.27 회의, 좀비 분해 효과 공부

1.28 ~ 1.30 분해 효과 공부, 좀비 팔다리 떨어지는 기능 구현

아래 사진은 기본적인 총알의 데미지 전달 메커니즘이다.

총알이 무언가에 부딪히면

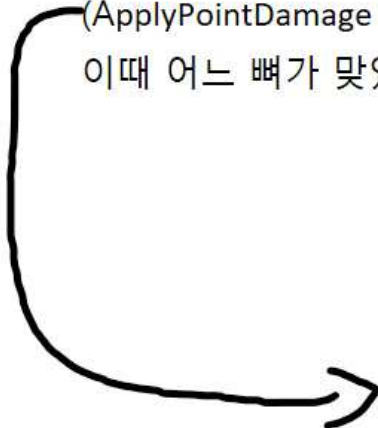


On Hit 함수 호출

여기서 엔진에게 엑터에게 데미지를 가해줘 하고

(ApplyPointDamage 함수 호출함)

이때 어느 뼈가 맞았는지 정보가 담긴 , FHitResult를 넘김



ApplyPointDamage 함수가 호출되면

엔진은 다른 Actor에게

OnTakePointDamage 델리게이트를 발생시킴

이번 주 한 일

UHealthComponent 코드를 보면 게임 시작 시 이 이벤트를 듣겠다고 구독해 놓은 것을 알 수 있다.

```
void UHealthComponent::BeginPlay()
{
    Super::BeginPlay();

    GetOwner()->OnTakePointDamage.AddDynamic(this, &UHealthComponent::PointDamageTaken);
}
```

HealthComponent를 가진 Actor가 PointDamage를 입으면,
그 Actor의 PointDamageTaken 함수가 실행이 된다.

위와 같은 방법으로 좀비는 피해를 입을 때 어떤 뼈에 데미지를 입었는지 HitResult를 통하여 알 수 있게 되었다.

아래 사진과 같이 좀비의 스켈레톤 이름을 가져와, Key(키)와 Value(값)이 쌍으로 이루어진 map 자료구조를 이용하여 저장해 주었다.

뼈의 개수가 엄청 많지않지만,이름이 같은 뼈가 존재하지않고, 저장순서도 유지할 필요가 없으므로 map을 사용하여 뼈의 이름과 내구도를 관리하면,
피격된 뼈를 효과적으로 찾을 수 있을거라 생각해서 map을 사용했다.

```
void ABaseZombie::InitializeBoneDurability()
{
    BoneDurability.Add(FName("head"), 20.0f);

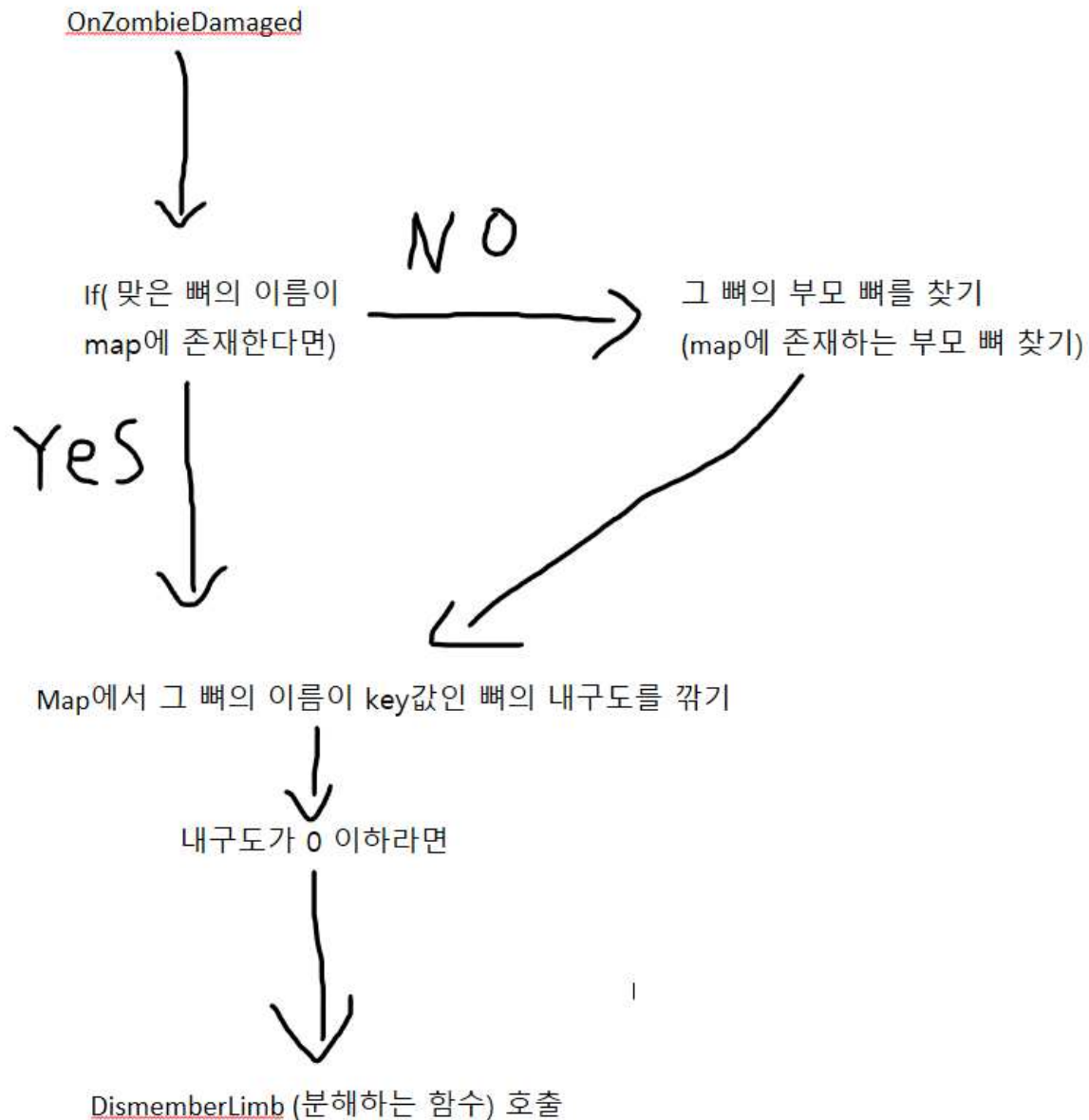
    // 팔
    BoneDurability.Add(FName("upperarm_l"), 15.0f);
    BoneDurability.Add(FName("lowerarm_l"), 10.0f);
    BoneDurability.Add(FName("upperarm_r"), 15.0f);
    BoneDurability.Add(FName("lowerarm_r"), 10.0f);

    // 다리
    BoneDurability.Add(FName("thigh_l"), 20.0f);
    BoneDurability.Add(FName("calf_l"), 15.0f);
    BoneDurability.Add(FName("thigh_r"), 20.0f);
    BoneDurability.Add(FName("calf_r"), 15.0f);

    // 척추
    BoneDurability.Add(FName("spine_01"), 30.0f);
}
```

이번 주 한 일

아래 사진은 좀비 팔,다리 분해 메커니즘이다.



아래는 분해하는 기능을 하는 `DismemberLimb` 함수 사진이다.

```
void ABaseZombie::DismemberLimb(FName BoneName, FVector Impulse, FVector HitLocation)
{
    if (BrokenBones.Contains(BoneName)) return;

    // 부러진 뼈라고 기록해놓기
    BrokenBones.Add(BoneName);

    if (ZombieMesh)
    {
        // 제약 조건 파괴 (뼈를 물리적으로 분리)
        ZombieMesh->BreakConstraint(Impulse, HitLocation, BoneName);

        // 잘린 부위가 물리 시뮬레이션을 하도록 설정
        // 이 설정이 없으면 잘린 팔이 공중에 둥둥 떠다니며 애니메이션을 계속 따라함.
        ZombieMesh->SetAllBodiesBelowSimulatePhysics(BoneName, true, true);

        // 물리 충격 가하기 (잘려나갈 때 튀어나가도록)
        ZombieMesh->AddImpulse(Impulse, BoneName, true);

        UE_LOG(LogTemp, Warning, TEXT("Dismembered: %s"), *BoneName.ToString());
    }
}
```

이번 주 한 일

BreakConstraint 함수를 사용해서 뼈의 제약 조건을 파괴하는 방식을 사용하여 신체 분리를 해주었다.

위와 같은 방식으로 구현하였는데, 총알을 쏘도 아무런 일이 일어나지 않아. 디버깅을 해보니 HitResult에서 뼈의 이름이 자꾸 None이라 뜨는 문제가 발생하였다.

왜 그런지 원인을 찾아보니, 총알이 메쉬보다 캡슐에 먼저 맞아 생기는 문제였다.

문제를 해결하기 위해, 캐릭터 외곽을 감싸는 캡슐 콜리전에 의한 간접 문제를 해결하기 위해, 총알과 캡슐 간의 충돌 채널을 분리하였습니다.

이를 통해 캡슐은 통과하되 메쉬 표면에서만 충돌이 발생하도록 물리 필터링을 설계하였으며, 결과적으로 정밀한 부위별 피격 판정 및 신체 분리 로직을 완성하였습니다.

이런식으로 문제를 해결하고, 실행을 해보니, 뼈의 이름이 정상적으로 뜨고, 팔이 분해가 되었지만, 아래 사진과 같이 메쉬가 늘어나는 문제가 발생하였습니다.



해결 방안을 고민하는 중입니다.

다음 주 할 일

- 좀비 팔 다리 분해과정에 생긴 문제 해결하기.
- 문제 해결하고, 구매한 좀비 에셋 프로젝트에 migrate하여 적용해보기.

특이사항

4주 차 보고서

(2026년 1 월 25 일 ~ 2026년 1 월 31 일)

작성자

안윤진

이번 주 한 일

이번주는 다른 일정으로 인해 정말 조금밖에 하지 못했다.



하나하나 레벨에 직접 배치한 버전



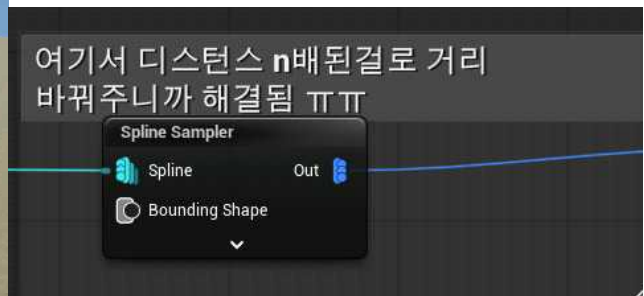
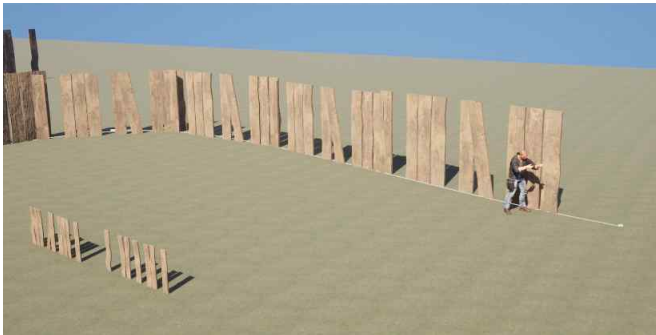
hism을 사용한 액터를 배치한 버전

들을 비교하였을 때 별반 차이가 나지 않았다. 이는 hims으로 사용된 메시가 너무 적기에 진가를 발휘하지 못한 것으로 판단해, 앞으로 집을 더 많이 놓고 수동배치 vs hims버전으로 비교해 볼 것 이다.

이번 주 한 일



처음에 맵1의 탈출 부분을 이렇게 구성하였는데, 팀 회의 결과 철조망이 아닌 울타리로 통일감있게 구성하는 것이 좋겠다하여, pcg의 메쉬를 변경하였다. 이 과정에서 메쉬들이 자꾸 겹치는 현상이 발생해, 식도 바뀌고, 피벗들도 변경해보았는데 그냥 내가 과거에 수동세팅해둔 부분이 문제였었다. 지금은 정상적으로 그려진다. 그리고 이때 울타리 메쉬 디자인 다른 3가지를 pcg에 있는 기능으로 각각의 가중치를 통해 랜덤 배치되게 하였다.



여기서 디스턴스 n배된걸로 거리 바꿔주니까 해결됨 ㅠㅠ

PCG 울타리 수정 → 3종류 매치 랜덤배치. (가중치 통해 선택가능)

→ 디스턴스 통해 연결 거리 수정.

노다 집 왔던 중, 집 틀이 왔으면 좋겠다 생각.

현재 집 BP 제작 중.

Class에서 수정으로 세팅해줌, 6칸바다도...

→ 개선방안.

전부 변형 BP로 제작하기.

바닥, $N \times M$, n, m 변형. / 용하지 않는다.

벽, 그 외 것들 전부 → 완전한 변형으로 생성가능하게.

가중치 변형. (가중치) →

→ 울타리 랜스케이드 변경 후 이렇게 잘 따라가는지 확인.

→ 편집 노다 생성 후 환경, HSM 생성 후 환경 비교 캡처.

✓

1차 이렇게 노다 vs HSM 샘플터 → 별 차이 없음.

why? 매쉬 6개 정도는 HSM 하나하나.. 여러 개 변형이 있음.

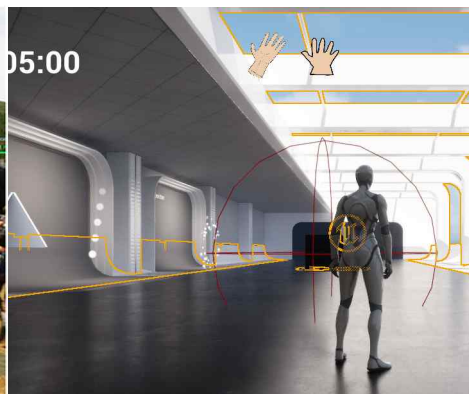
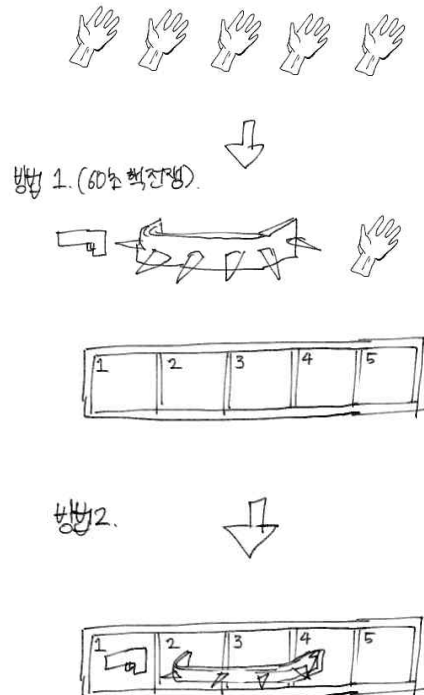
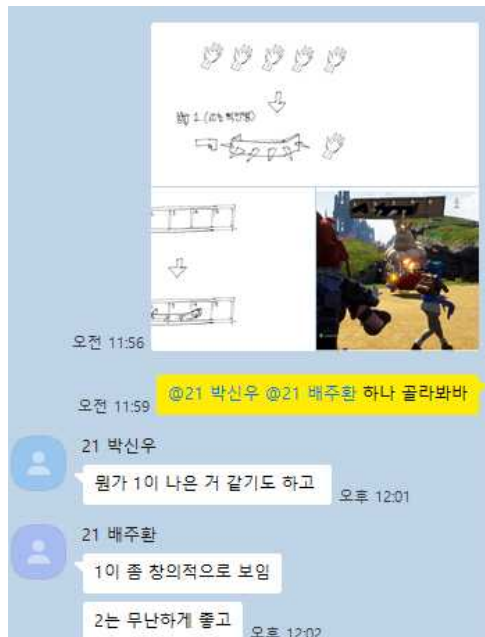
집 6개는 한 뒤 비교해봄.

→ UI 인벤트리 디자인 제작.

이번 주 한 일



그 외 이런저런 집 메쉬를 수동 배치하였고, 팀원들의 의견을 받고,



피드백을 통해 인벤토리 형식 ui로 사용될 손을 그렸다.

다음 주 할 일

월요일 - 맵에 수동 배치 집 3개 이상 디자인 추가, 개선한 집 베이스 class 제작
화,수 - 집 블루프린트 액터로 변경해 제작
목 - 인벤토리 UI 제작 및 기존 timer ui랑 겹쳐 배치하기
금 - 약속 일정
토 - 이 부분에서 부족한 점들 빠르게 하기

특이사항

주마다의 할 일을 잘 지키지 않고 있음. 2월부터 정신각성 필요

5주 차 보고서

(2025년 1 월 25 일 ~ 2026년 1 월 31 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

프레임 워크에 Lock과 DeadLockProfiler를 추가했다. 멀티쓰레드 환경에서 데이터의 안정성을 보장하기 위해 사용하는 RW SpinLock을 사용하여 구현해보았다. 동시 읽기는 허용하되, 쓰기는 독점적으로 제어하여 데이터 일관성을 지키는 장치를 만들었다.

락을 추가했으니 이 락들이 서로 꼬여서 발생하는 데드락 문제가 빈번하게 발생할 수 있는데 이때 데드락을 사전에 감지하고 예방하기 위해 DeadLockProfiler를 만들었다. PushLock은 락을 획득할 때 호출하며 현재 잡고 있는 락 -> 지금 잡으려는 락의 관계를 저장하고 처음 보는 순서라면 사이클 검사를 요청한다. PopLock은 락을 해제할 때 호출한다. 현재 잡고 있는 락 목록에서 제거하여 순서 추적을 유지한다. CheckCycle은 그래프에 순환이 있는지 검사를 하고 Dfs는 그래프를 파고들며 순환 구조가 있는지 찾아내는 알고리즘이다.

메모리에 관한 내용을 추가로 공부했다.

```

/*-----
:   RefCountable
-----*/

class RefCountable
{
public:
    RefCountable() : _refCount(1) {}
    virtual ~RefCountable() {}

    int32 GetRefCount() { return _refCount; }

    int32 AddRef() { return ++_refCount; }
    int32 ReleaseRef()
    {
        int32 refCount = --_refCount;
        if (_refCount == 0)
        {
            delete this;
        }
        return refCount;
    }

protected:
    atomic<int32> _refCount;
};

```

New로 생성한 객체를 적절한 시점에 delete하지 않으면 메모리 누수가 발생하는데 RefCountable을 만들어서 사용하는 곳이 아무곳도 없을 때 스스로 파괴하도록 자동화 시스템을 만들었다.

```

template<typename Type, typename... Args>
Type* xnew(Args&&... args)
{
    Type* memory = static_cast<Type*>(xalloc(sizeof(Type)));
    new(memory)Type(forward<Args>(args)...);    // placement new
    return memory;
}

template<typename Type>
void xdelete(Type* obj)
{
    obj->~Type();
    xrelease(obj);
}

```

```

#ifdef _DEBUG
#define xalloc(size)      BaseAllocator::Alloc(size)
#define xrelease(ptr)    BaseAllocator::Release(ptr)
#else
#define xalloc(size)      BaseAllocator::Alloc(size)
#define xrelease(ptr)    BaseAllocator::Release(ptr)
#endif

```

C++의 기본연산자인 `new` 대신에 `xnew`를 만들어서 수동으로 정밀하게 제어하는 코드를 만들었다. `xalloc`으로 순수 메모리 공간을 확보하고, 해당 위치에 객체 생성자를 강제로 호출한 후, 완성된 객체 포인터를 반환한다. `xdelete`는 객체의 소멸자를 명시적으로 호출하여 자원을 정리하고, `xrelease`를 통해 실제 메모리 공간을 해제한다.

```

class BaseAllocator
{
public:
    static void* Alloc(int32 size);
    static void Release(void* ptr);
};

```

```

void* BaseAllocator::Alloc(int32 size)
{
    return ::malloc(size);
}

void BaseAllocator::Release(void* ptr)
{
    ::free(ptr);
}

```

```

template<typename T>
class TSharedPtr
{
public:
    TSharedPtr() {}
    TSharedPtr(T* ptr) { Set(ptr); }

    // 복사
    TSharedPtr(const TSharedPtr& rhs) { Set(rhs._ptr); }
    // 이동
    TSharedPtr(TSharedPtr&& rhs) { _ptr = rhs._ptr; rhs._ptr = nullptr; }
    // 상속 관계 복사
    template<typename U>
    TSharedPtr(const TSharedPtr<U>& rhs) { Set(static_cast<T*>(rhs._ptr)); }

    ~TSharedPtr() { Release(); }

public:
    // 복사 연산자
    TSharedPtr& operator=(const TSharedPtr& rhs)
    {
        if (_ptr != rhs._ptr) {
            Release();
            Set(rhs._ptr);
        }
        return *this;
    }
}

```

```

// 이동 연산자
TSharedPtr& operator=(TSharedPtr&& rhs)
{
    Release();
    _ptr = rhs._ptr;
    rhs._ptr = nullptr;
    return *this;
}

bool operator==(const TSharedPtr& rhs) const { return _ptr == rhs._ptr; }
bool operator==(T* ptr) const { return _ptr == ptr; }
bool operator!=(const TSharedPtr& rhs) const { return _ptr != rhs._ptr; }
bool operator!=(T* ptr) const { return _ptr != ptr; }
bool operator<(const TSharedPtr& rhs) const { return _ptr < rhs._ptr; }
T* operator*() { return _ptr; }
const T* operator*() const { return _ptr; }
operator T* () const { return _ptr; }
T* operator->() { return _ptr; }
const T* operator->() const { return _ptr; }

bool IsNull() { return _ptr == nullptr; }

```

```

private:
    inline void Set(T* ptr)
    {
        _ptr = ptr;
        if (ptr)
            ptr->AddRef();
    }

    inline void Release()
    {
        if (_ptr != nullptr)
        {
            _ptr->ReleaseRef();
            _ptr = nullptr;
        }
    }

private:
    T* _ptr = nullptr;
};

```

```

class Wraight : public RefCountable
{
public:
    int _hp = 150;
    int _posX = 0;
    int _posY = 0;
};

using WraightRef = TSharedPtr<Wraight>;

class Missile : public RefCountable
{
public:
    void SetTarget(WraightRef target)
    {
        _target = target;
        // 중간에 개입 가능
        //target->AddRef();
    }

    bool Update()
    {
        if (_target == nullptr)
            return true;

        int posX = _target->_posX;
        int posY = _target->_posY;

        // TODO : 쫓아간다

        if (_target->_hp == 0)
        {
            _target->ReleaseRef();
            _target = nullptr;
            return true;
        }

        return false;
    }

    WraightRef _target = nullptr;
};

using MissileRef = TSharedPtr<Missile>;

```

```

int main()
{
    WraightRef wraight(new Wraight());
    wraight->ReleaseRef();
    MissileRef missile(new Missile());
    missile->ReleaseRef();

    missile->SetTarget(wraight);

    // 레이스가 피격 당함
    wraight->_hp = 0;
    //wraight->ReleaseRef();
    //delete wraight;
    wraight = nullptr;

    while (true)
    {
        if (missile)
        {
            if (missile->Update())
            {
                missile->ReleaseRef();
                missile = nullptr;
            }
        }

        //missile->ReleaseRef();
        //delete missile;
        missile = nullptr;
    }
}

```

앞서 만든 RefCountable 시스템을 실제로 사용하기 위한 스마트 포인터 구현체와 활용한 코드이다. C++의 shared_ptr과 유사하게 TSharedPtr<T>를 만들었다.

```

/*-----
| STLAllocator
|-----*/

template<typename T>
class StlAllocator
{
public:
    using value_type = T;

    StlAllocator() {}

    template<typename Other>
    StlAllocator(const StlAllocator<Other>&) {}

    T* allocate(size_t count)
    {
        const int32 size = static_cast<int32>(count * sizeof(T));
        return static_cast<T*>(Xalloc(size));
    }

    void deallocate(T* ptr, size_t count)
    {
        xrelease(ptr);
    }
};

```

```

template<typename Type>
using Vector = vector<Type, StlAllocator<Type>>;

template<typename Type>
using List = list<Type, StlAllocator<Type>>;

template<typename Key, typename Type, typename Pred = less<Key>>
using Map = map<Key, Type, Pred, StlAllocator<pair<const Key, Type>>>;

template<typename Key, typename Pred = less<Key>>
using Set = set<Key, Pred, StlAllocator<Key>>;

template<typename Type>
using Deque = deque<Type, StlAllocator<Type>>;

template<typename Type, typename Container = Deque<Type>>
using Queue = queue<Type, Container>;

template<typename Type, typename Container = Deque<Type>>
using Stack = stack<Type, Container>;

template<typename Type, typename Container = Vector<Type>, typename Pred = less<typename Container::value_type>>
using PriorityQueue = priority_queue<Type, Container, Pred>;

using String = basic_string<char, char_traits<char>, StlAllocator<char>>;
using WString = basic_string<wchar_t, char_traits<wchar_t>, StlAllocator<wchar_t>>;

template<typename Key, typename Type, typename Hasher = hash<Key>, typename KeyEq = equal_to<Key>>
using HashMap = unordered_map<Key, Type, Hasher, KeyEq, StlAllocator<pair<const Key, Type>>>;

template<typename Key, typename Type, typename Hasher = hash<Key>, typename KeyEq = equal_to<Key>>
using HashSet = unordered_set<Key, Hasher, KeyEq, StlAllocator<Key>>;

```

C++ 표준 라이브러리인 STL 컨테이너가 `xalloc`, `xrelease`를 사용하도록 연결해주는 어댑터 역할을 하는 클래스이다. `StlAllocator`의 핵심 역할은 STL이 메모리가 필요할때마다 표준 할당자 대신 `xalloc`을 호출하게끔 통로를 변경해준다.

```
class StompAllocator
{
    enum {PAGE_SIZE = 0x1000 };
public:
    static void* Alloc(int32 size);
    static void Release(void* ptr);
};

void* StompAllocator::Alloc(int32 size)
{
    const int64 pageCount = (size + PAGE_SIZE - 1) / PAGE_SIZE;
    const int64 dataOffset = pageCount * PAGE_SIZE - size;
    void* baseAddress = ::VirtualAlloc(NULL, pageCount * PAGE_SIZE, MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
    return static_cast<void*>(static_cast<int8*>(baseAddress) + dataOffset);
}

void StompAllocator::Release(void* ptr)
{
    const int64 address = reinterpret_cast<int64*>(ptr);
    const int64 baseAddress = address - (address % PAGE_SIZE);
    ::VirtualFree(reinterpret_cast<void*>(baseAddress), 0, MEM_RELEASE);
}
```

`StompAllocator`는 메모리 오염 및 오버런을 즉각적으로 잡아내는 디버깅에 특화되어있다. OS의 가상 메모리를 직접 컨트롤 해서 메모리 범위가 조금이라도 벗어나서 읽거나 쓰는 순간 프로그램이 크래쉬나게 한다.

다음 주 할 일

메모리에 대한 공부를 많이 했는데 여태까지 공부한 것들은 성능 면에서는 좋지 않았다. OS 커널까지 가야하기 때문이다. 다음 주에는 성능을 극대화하기 위해 많이 사용하는 메모리 풀에 대해서 공부할 예정이다.

특이사항